

```
import marimo as mo
```

EE446 - TinyML - Assignment 2

Importing Libraries

```
import pandas as pd
import numpy as np
from sklearn.metrics import confusion_matrix, classification_report
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
import tensorflow as tf
import seaborn as sns
from pylab import rcParams
from sklearn.model_selection import train_test_split
from tensorflow.keras import datasets, layers, models
from tensorflow.keras.models import Model, load_model, Sequential
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.layers import Input, Dense, Activation
from tensorflow.keras.callbacks import ModelCheckpoint, TensorBoard
from tensorflow.keras import regularizers
from sklearn.preprocessing import LabelEncoder
import warnings
warnings.filterwarnings("ignore")
from sklearn.utils import class_weight
from sklearn.metrics import accuracy_score
import c_writer
from os.path import join
```

```
2026-08-12 23:07:00.315286: I tensorflow/core/util/port.cc:111] oneDNN custom operations are on. You may
2026-08-12 23:07:00.336486: E tensorflow/compiler/xla/stream_executor/cuda/cuda_dnn.cc:9342] Unable to i
2026-08-12 23:07:00.336512: E tensorflow/compiler/xla/stream_executor/cuda/cuda_fft.cc:609] Unable to r
2026-08-12 23:07:00.336533: E tensorflow/compiler/xla/stream_executor/cuda/cuda_blas.cc:1518] Unable to
2026-08-12 23:07:00.341227: I tensorflow/core/platform/cpu_feature_guard.cc:182] This TensorFlow binary
To enable the following instructions: AVX2 AVX512F AVX512_VNNI AVX512_BF16 FMA, in other operations, rel
```

Reading Data

```
# Reading the data and adding column header (feature) names
data = pd.read_csv("Network_anomaly_data.txt", sep=",", names=["duration", "protocoltype", "service",
"flag", "srcbytes", "dstbytes", "land", "wrongfragment", "urgent", "hot", "numfailedlogins", "loggedin", "numc
"rootshell", "suattempted", "numroot", "numfilecreations", "numshells", "numaccessfiles", "numoutboundcmds",
"isguestlogin", "count", "srvcount", "serrorate", "srvserrorate", "rerrorate", "srvrerrorate", "samesrvra
"diffsrvrate", "srvdiffhostrate", "dsthostcount", "dsthostsrvcount", "dsthostsamesrvrate", "dsthostdiffsrv
"dsthostsamesrcpportrate", "dsthostsrvdiffhostrate", "dsthosterrorate", "dsthostsrvserrorate", "dsthostre
"dsthostsrvrerrorate", "attack", "lastflag"])

data # printing the dataframe
```

Question 1: Data Preprocessing

(a) Drop the 'land', 'urgent', 'numfailedlogins', 'numoutboundcmds' columns from the dataframe "data".

```
# (a) drop the four requested columns in place
data.drop(['land', 'urgent', 'numfailedlogins', 'numoutboundcmds'], axis=1, inplace=True)
```

```
print(data.shape)
(125973, 39)
```

(b) Change any label that is not named normal to attack in the {'attack'} column of the dataframe data.

```
# (b) binarize the label: everything that is not "normal" becomes "attack"
data.loc[data['attack'] != 'normal', 'attack'] = 'attack'
cat_cols = ['protocoltype', 'service', 'flag', 'attack']
print(data['attack'].value_counts())

attack
normal      67343
attack      58630
Name: count, dtype: int64

data
```

(c) Use LabelEncoder() function from the sklearn.preprocessing library to convert non-numerical attributes in the {'protocoltype', 'service', 'flag', 'attack'} columns of the dataframe data to numerical values.

```
# (c) label-encode the non-numerical columns (attack -> 0, normal -> 1)
for _col in cat_cols:
    data[_col] = LabelEncoder().fit_transform(data[_col])
print(data[cat_cols].dtypes)

protocoltype    int64
service         int64
flag           int64
attack          int64
dtype: object

pd.DataFrame(data) #<----- Print your modified dataframe "data"
```

Feature Scaling and Train/Test Split

```
# All the features apart from Attack are what we are going to use to predict the attack status of the d
# attack = 1 (normal/not an attack) and attack = 0 (attack)
X = data.drop(['attack'],axis=1).to_numpy()

# Initialize the StandardScaler
scaler = StandardScaler()

# Fit and transform the data
X_normalized = scaler.fit_transform(X)

Y = data['attack'].to_numpy()
# Splitting X and y testing and training data
# we are taking 20% of the data for testing and 80% of the data for training
X_train, X_test, y_train, y_test = train_test_split(X_normalized, Y, test_size = 0.20)
# reshaping y test and train array
y_train = y_train.reshape(len(y_train),1)
y_test = y_test.reshape(len(y_test),1)
```

Question 2: Dimensionality Reduction for Visualization

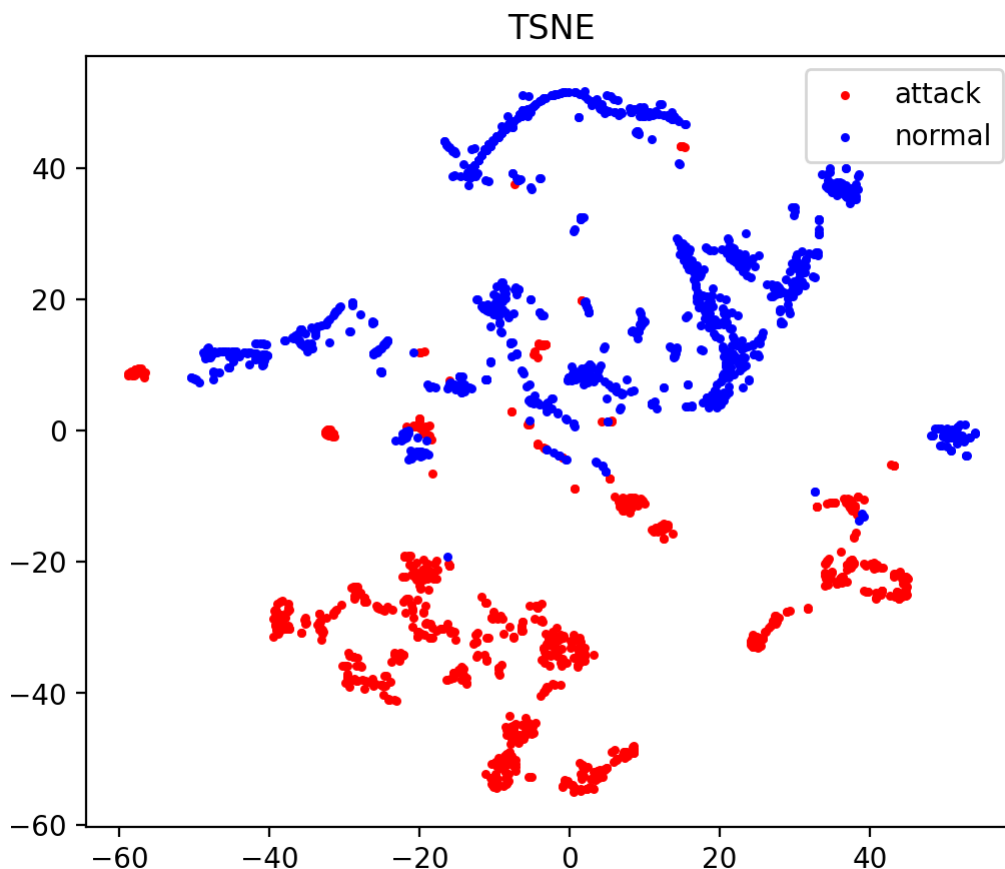
(a) Use TSNE from the `sklearn.manifold` library to visualize the data in the test set (`X_test`) in 2D. In your figure, use color "red" to mark {attack} data points and color "blue" to mark {normal} data points.

```
from sklearn.manifold import TSNE

# t-SNE and kernel PCA are  $O(n^2)$ , so use a random 2000-point subsample of the test set
_rng = np.random.default_rng(0)
sub = _rng.choice(len(X_test), 2000, replace=False)
X_sub, y_sub = X_test[sub], y_test[sub].ravel()

def plot_2d(Z, title):
    fig, ax = plt.subplots(figsize=(6, 5))
    ax.scatter(Z[y_sub == 0, 0], Z[y_sub == 0, 1], c='red', s=5, label='attack')
    ax.scatter(Z[y_sub == 1, 0], Z[y_sub == 1, 1], c='blue', s=5, label='normal')
    ax.set_title(title)
    ax.legend()
    fig.savefig('../submission/' + title.lower().replace('-', ' ') + '.png', dpi=120, bbox_inches='tight')
    return fig

plot_2d(TSNE(n_components=2, random_state=0).fit_transform(X_sub), 'TSNE')
```

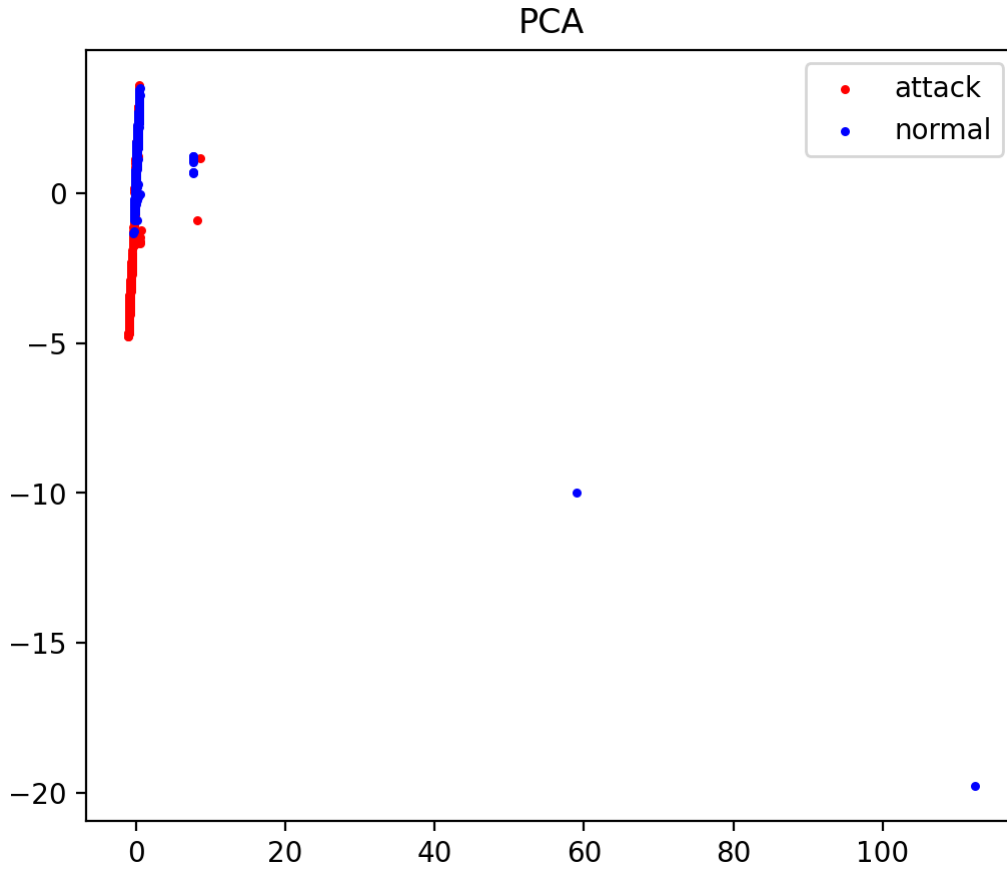


(b) Use PCA from the `sklearn.decomposition` library to visualize the data in the test set (`X_test`) in 2D. In your figure, use color "red" to mark {attack} data points and color "blue"

to mark {normal} data points.

```
from sklearn.decomposition import PCA
```

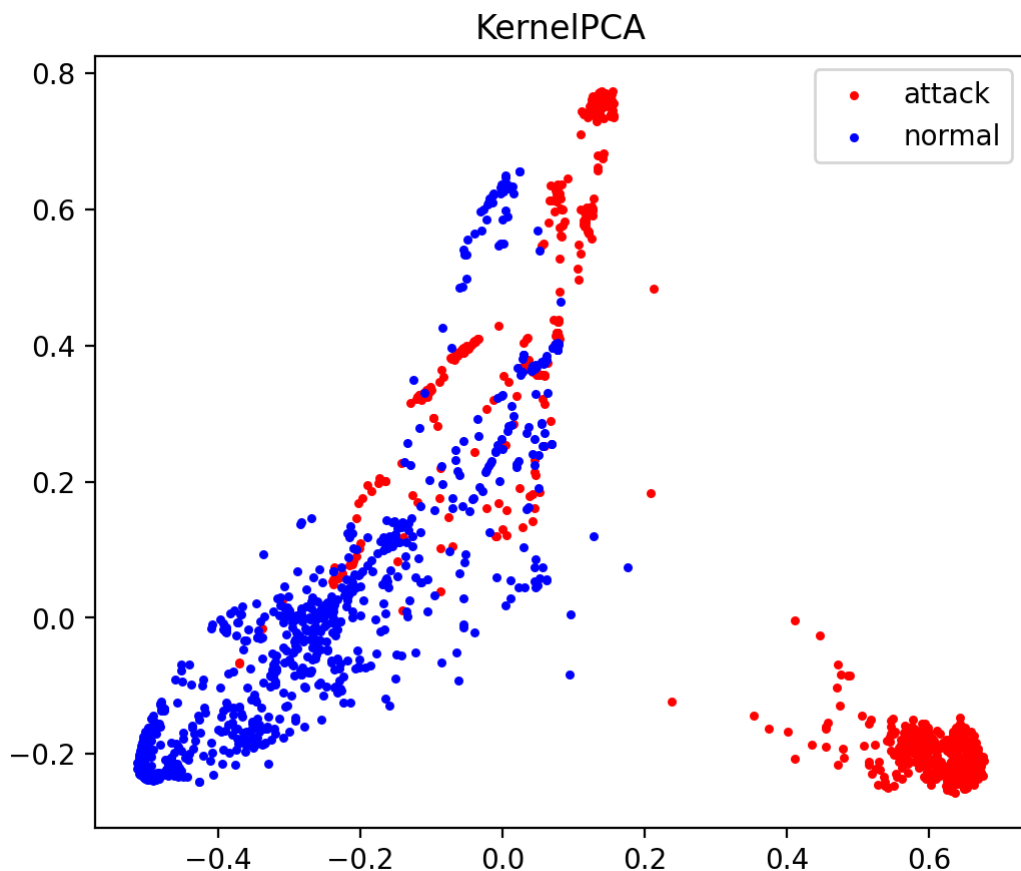
```
plot_2d(PCA(n_components=2).fit_transform(X_sub), 'PCA')
```



(c) Use `KernelPCA` from the `sklearn.decomposition` library to visualize the data in the test set (`X_test`) in 2D. Use radial basis function (`rbf`) as the kernel. In your figure, use color "red" to mark {attack} data points and color "blue" to mark {normal} data points.

```
from sklearn.decomposition import KernelPCA
```

```
plot_2d(KernelPCA(n_components=2, kernel='rbf').fit_transform(X_sub), 'KernelPCA')
```



Question 3: Implementing a DNN on the dataset

(a) Implement a deep neural network (DNN) on the Network Anomaly Dataset. Ensure that the output layer contains one neuron with sigmoid activation.

Define the DNN model

```
base_model = Sequential([
    Dense(64, activation='relu', input_shape=(X_train.shape[1],)),
    Dense(32, activation='relu'),
    Dense(16, activation='relu'),
    Dense(1, activation='sigmoid')
])
base_model.summary()
```

2026-08-12 23:07:05.892286: I tensorflow/core/common_runtime/gpu/gpu_device.cc:1886] Created device /job

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 64)	2496
dense_1 (Dense)	(None, 32)	2080
dense_2 (Dense)	(None, 16)	528

dense_3 (Dense) (None, 1) 17

```
=====
Total params: 5121 (20.00 KB)
Trainable params: 5121 (20.00 KB)
Non-trainable params: 0 (0.00 Byte)
-----
```

(b) Compile and train your DNN model on the training set (`X_train`). Denote the trained model as `base_model`.

```
base_model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
base_model.fit(X_train, y_train, epochs=10, batch_size=256, validation_split=0.1, verbose=2)
```

Epoch 1/10

```
2026-08-12 23:07:06.959278: I tensorflow/tsl/platform/default/subprocess.cc:304] Start cannot spawn child
2026-08-12 23:07:07.353930: I tensorflow/compiler/xla/service/service.cc:168] XLA service 0x7f3f703abc8
2026-08-12 23:07:07.353951: I tensorflow/compiler/xla/service/service.cc:176] StreamExecutor device (0:
2026-08-12 23:07:07.357987: I tensorflow/compiler/mlir/tensorflow/utils/dump_mlir_util.cc:269] disabling
2026-08-12 23:07:07.371124: I tensorflow/compiler/xla/stream_executor/cuda/cuda_dnn.cc:442] Loaded cuDNN
2026-08-12 23:07:07.425646: I ./tensorflow/compiler/jit/device_compiler.h:186] Compiled cluster using XLA
```

```
355/355 - 3s - loss: 0.0826 - accuracy: 0.9726 - val_loss: 0.0196 - val_accuracy: 0.9934 - 3s/epoch - 9
```

Epoch 2/10

```
355/355 - 2s - loss: 0.0147 - accuracy: 0.9955 - val_loss: 0.0118 - val_accuracy: 0.9952 - 2s/epoch - 5
```

Epoch 3/10

```
355/355 - 1s - loss: 0.0098 - accuracy: 0.9965 - val_loss: 0.0110 - val_accuracy: 0.9964 - 977ms/epoch - 3
```

Epoch 4/10

```
355/355 - 1s - loss: 0.0081 - accuracy: 0.9970 - val_loss: 0.0093 - val_accuracy: 0.9967 - 1s/epoch - 3
```

Epoch 5/10

```
355/355 - 1s - loss: 0.0073 - accuracy: 0.9971 - val_loss: 0.0090 - val_accuracy: 0.9968 - 1s/epoch - 4
```

Epoch 6/10

```
355/355 - 1s - loss: 0.0061 - accuracy: 0.9977 - val_loss: 0.0076 - val_accuracy: 0.9976 - 920ms/epoch - 3
```

Epoch 7/10

```
355/355 - 1s - loss: 0.0055 - accuracy: 0.9980 - val_loss: 0.0074 - val_accuracy: 0.9967 - 927ms/epoch - 3
```

Epoch 8/10

```
355/355 - 1s - loss: 0.0055 - accuracy: 0.9979 - val_loss: 0.0080 - val_accuracy: 0.9967 - 900ms/epoch - 3
```

Epoch 9/10

```
355/355 - 1s - loss: 0.0056 - accuracy: 0.9980 - val_loss: 0.0060 - val_accuracy: 0.9978 - 906ms/epoch - 3
```

Epoch 10/10

```
355/355 - 1s - loss: 0.0043 - accuracy: 0.9985 - val_loss: 0.0051 - val_accuracy: 0.9984 - 886ms/epoch - 3
```

(c) Evaluate the `base_model` on the test set (`X_test`) using `classification_report` and `confusion_matrix` from the `sklearn.metrics` library. Report these numbers in your .pdf writeup file using screenshots.

```
y_pred = (base_model.predict(X_test, verbose=0) > 0.5).astype(int)
print(classification_report(y_test, y_pred, target_names=['attack', 'normal']))
print(confusion_matrix(y_test, y_pred))
```

	precision	recall	f1-score	support
attack	1.00	1.00	1.00	11688
normal	1.00	1.00	1.00	13507
accuracy			1.00	25195

macro avg	1.00	1.00	1.00	25195
weighted avg	1.00	1.00	1.00	25195

```
[[11680      8]
 [   30 13477]]
```

```
# Save the original Keras model to HDF5 file
base_model.save('original_model.h5')
```

Question 4: Implementing Quantized Model

(a) Implement full-integer INT8 post-training quantization on the `base_model`. Use a representative dataset from `X_train` to calibrate the activation ranges. Configure the converter to use INT8 operators with INT8 input and output tensors. Designate the resulting quantized model as `tflite_quant_model`.

```
# Load the trained model
base_model_1 = tf.keras.models.load_model('original_model.h5')
```

```
def representative_dataset():
    # calibrate activation ranges on a slice of the training set
    for _x in X_train[:500].astype(np.float32):
        yield [_x.reshape(1, -1)]
```

```
converter = tf.lite.TFLiteConverter.from_keras_model(base_model_1)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
converter.representative_dataset = representative_dataset
converter.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
converter.inference_input_type = tf.int8
converter.inference_output_type = tf.int8
tflite_quant_model = converter.convert()
```

```
2026-08-12 23:07:21.348338: W tensorflow/compiler/mlir/lite/python/tf_tfl_flatbuffer_helpers.cc:378] Ignored
2026-08-12 23:07:21.348368: W tensorflow/compiler/mlir/lite/python/tf_tfl_flatbuffer_helpers.cc:381] Ignored
2026-08-12 23:07:21.348726: I tensorflow/cc/saved_model/reader.cc:83] Reading SavedModel from: /tmp/tmpf...
2026-08-12 23:07:21.349638: I tensorflow/cc/saved_model/reader.cc:51] Reading meta graph with tags { se...
2026-08-12 23:07:21.349647: I tensorflow/cc/saved_model/reader.cc:146] Reading SavedModel debug info (i...
2026-08-12 23:07:21.351785: I tensorflow/compiler/mlir/mlir_graph_optimization_pass.cc:382] MLIR V1 opti...
2026-08-12 23:07:21.352263: I tensorflow/cc/saved_model/loader.cc:233] Restoring SavedModel bundle.
2026-08-12 23:07:21.377982: I tensorflow/cc/saved_model/loader.cc:217] Running initialization op on Saved...
2026-08-12 23:07:21.383969: I tensorflow/cc/saved_model/loader.cc:316] SavedModel load for tags { serve...
fully_quantize: 0, inference_type: 6, input_inference_type: INT8, output_inference_type: INT8
```

```
import os
```

```
# Save the quantized model
with open('quantized_model.tflite', 'wb') as f:
    f.write(tflite_quant_model)
```

```
# Get the file sizes
original_model_size = os.path.getsize('original_model.h5')
quantized_model_size = os.path.getsize('quantized_model.tflite')
```

```
# Print the model sizes
print(f"Original model size: {original_model_size / 1024:.2f} KB")
print(f"Quantized model size: {quantized_model_size / 1024:.2f} KB")
```

Original model size: 100.01 KB
Quantized model size: 8.24 KB

(b) Evaluate the `tflite_quant_model` on the test set (`X_test`) using `classification_report` and `confusion_matrix` from the `sklearn.metrics` library. Report these numbers in your .pdf writeup file using screenshots.

```
interpreter = tf.lite.Interpreter(model_content=tflite_quant_model)
interpreter.allocate_tensors()
in_det = interpreter.get_input_details()[0]
out_det = interpreter.get_output_details()[0]
in_scale, in_zp = in_det['quantization']
out_scale, out_zp = out_det['quantization']
print(f"input scale={in_scale}, zero_point={in_zp}")

y_pred_quant = np.empty((len(X_test), 1), dtype=int)
for _i, _x in enumerate(X_test):
    # quantize the sample with the input tensor's scale and zero point
    _xq = np.clip(np.round(_x / in_scale) + in_zp, -128, 127).astype(np.int8)
    interpreter.set_tensor(in_det['index'], _xq.reshape(1, -1))
    interpreter.invoke()
    _out = interpreter.get_tensor(out_det['index'])[0][0]
    y_pred_quant[_i] = ((_out - out_zp) * out_scale) > 0.5

print(classification_report(y_test, y_pred_quant, target_names=['attack', 'normal']))
print(confusion_matrix(y_test, y_pred_quant))

input scale=0.11214608699083328, zero_point=-52
INFO: Created TensorFlow Lite XNNPACK delegate for CPU.
```

	precision	recall	f1-score	support
attack	1.00	1.00	1.00	11688
normal	1.00	1.00	1.00	13507
accuracy			1.00	25195
macro avg	1.00	1.00	1.00	25195
weighted avg	1.00	1.00	1.00	25195

```
[[11681    7]
 [  42 13465]]
```

Converting `tflite_quant_model` to C and creating the header file

```
# c_writer is a py file in the same folder and has been imported at the beginning of the notebook
# Reference : https://github.com/ShawnHymel/tinymml-example-anomaly-detection/blob/master/utils/c_writer
# We use #04x to pad the output to 2 digits with a 0x prefix
hex_array = [format(val, '#04x') for val in tflite_quant_model]
# Calling function to convert an array into a C string (requires Numpy)
# create_array(np_array, var_type, var_name, line_limit=80, indent=4)
c_model = c_writer.create_array(np.array(hex_array), 'unsigned char', "network_model")
# Calling Function to create a header file with given C code as a string
header_str = c_writer.create_header(c_model, "network_model")

#Writing to the header file
```



```
with open('network_model.h', 'w') as file:
    file.write(header_str)
```

Generating Samples for Inference on Arduino

Converting a sample piece of the X test and y test data to C (for the purpose of ino code (arduino))t
the sample and compare

```
Xtest = X_test[0:5,:]  
print(c_writer.create_array(Xtest,"float","X_test"))  
  
const unsigned int X_test_dim1 = 5;  
const unsigned int X_test_dim2 = 38;  
  
const float X_test[5][38] = {  
    -0.11024922321249885, -0.12470615670462065, 2.2496402596904983,  
    -0.7362346401101137, -0.00776224074056876, -0.004918644383724874,  
    -0.08948642202040107, -0.09507567152556495, -0.8092618187059747,  
    -0.011663642603760032, -0.036651869142258646, -0.024436507262009306,  
    -0.01238515036740332, -0.02618002418454278, -0.018609896340735923,  
    -0.04122119759327531, -0.0028174939213690777, -0.0975309439715147,  
    0.3221818209983339, -0.24420394625771188, 1.602663889932865,  
    1.6051037177847889, -0.37436223991967527, -0.37443160310530493,  
    -1.3214280114927377, -0.016929597707948767, -0.37455970440553465,  
    0.7343425609306344, -0.9543885148106975, -1.0719329252940557,  
    -0.17441734820805624, -0.4801968475158174, -0.28910340026287856,  
    1.6087590765792643, 1.6189552037455606, -0.38763462350750655,  
    -0.3763870260680415, -0.21997021516886575, -0.11024922321249885,  
    -0.12470615670462065, -0.13620543239909322, -0.7362346401101137,  
    -0.00776224074056876, -0.004918644383724874, -0.08948642202040107,  
    -0.09507567152556495, -0.8092618187059747, -0.011663642603760032,  
    -0.036651869142258646, -0.024436507262009306, -0.01238515036740332,  
    -0.02618002418454278, -0.018609896340735923, -0.04122119759327531,  
    -0.0028174939213690777, -0.0975309439715147, 0.5317738841070986,  
    -0.28550603517076306, 1.602663889932865, 1.6051037177847889,  
    -0.37436223991967527, -0.37443160310530493, -1.3896685914060654,  
    0.03852930781964016, -0.37455970440553465, 0.7343425609306344,  
    -0.9814882224260931, -1.0942072406165508, -0.06855302025511306,  
    -0.4801968475158174, -0.28910340026287856, 1.6087590765792643,  
    1.6189552037455606, -0.38763462350750655, -0.3763870260680415,  
    0.21642633142264128, -0.11024922321249885, -0.12470615670462065,  
    -0.07502990183269344, -0.7362346401101137, -0.00776224074056876,  
    -0.004918644383724874, -0.08948642202040107, -0.09507567152556495,  
    -0.8092618187059747, -0.011663642603760032, -0.036651869142258646,  
    -0.024436507262009306, -0.01238515036740332, -0.02618002418454278,  
    -0.018609896340735923, -0.04122119759327531, -0.0028174939213690777,  
    -0.0975309439715147, 0.4793758683299075, -0.037693501692456145,  
    1.602663889932865, 1.6051037177847889, -0.37436223991967527,  
    -0.37443160310530493, -1.0939594117816442, -0.07238850323553762,  
    -0.37455970440553465, 0.7343425609306344, -0.8188899767337195,  
    -0.9382870333590839, -0.17441734820805624, -0.4801968475158174,  
    -0.28910340026287856, 1.6087590765792643, 1.6189552037455606,  
    -0.38763462350750655, -0.3763870260680415, -0.21997021516886575,  
    -0.11024922321249885, -0.12470615670462065, -0.44208308523109213,  
    0.7511112872365361, -0.007716246560121601, -0.004565769314001138,
```

```

-0.08948642202040107, -0.09507567152556495, 1.2356940323701657,
-0.011663642603760032, -0.036651869142258646, -0.024436507262009306,
-0.01238515036740332, -0.02618002418454278, -0.018609896340735923,
-0.04122119759327531, -0.0028174939213690777, -0.0975309439715147,
-0.7257784945454897, -0.35434285002584825, -0.6372092679572258,
-0.6319290328885425, -0.37436223991967527, -0.37443160310530493,
0.7712831058493207, -0.349683030873482, 3.4741184775599305,
-0.8482258927040347, 1.2587542737799418, 1.0664013456654926,
-0.43907816809041417, -0.4478339589698615, -0.02258683123335326,
-0.6395319051152512, -0.6248707997445304, -0.38763462350750655,
-0.3763870260680415, 0.6528228780141483, -0.11024922321249885,
-0.12470615670462065, -1.2373649825942892, -2.2235805674567635,
-0.00776224074056876, -0.004918644383724874, -0.08948642202040107,
-0.09507567152556495, -0.8092618187059747, -0.011663642603760032,
-0.036651869142258646, -0.024436507262009306, -0.01238515036740332,
-0.02618002418454278, -0.018609896340735923, -0.04122119759327531,
-0.0028174939213690777, -0.0975309439715147, 1.2915451128763706,
-0.32680812408381416, -0.6372092679572258, -0.6319290328885425,
2.7464027950834815, 2.715364578035608, -1.4579091713193935,
-0.016929597707948767, -0.37455970440553465, 0.7343425609306344,
-1.0085879300414886, -1.1164815559390462, -0.06855302025511306,
-0.4801968475158174, -0.28910340026287856, -0.6395319051152512,
-0.6248707997445304, 2.8744095478204597, 2.753913722653446,
0.21642633142264128
};

ytest=y_test[0:5]
print(c_writer.create_array(ytest,"uint8_t","y_test"))

const unsigned int y_test_dim1 = 5;
const unsigned int y_test_dim2 = 1;

const uint8_t y_test[5][1] = {
    0, 0, 0, 1, 0
};

```

Question 5 (d): ten more samples, excluding the first five

```

Xtest10 = X_test[5:15,:]
print(c_writer.create_array(Xtest10,"float","X_test"))

const unsigned int X_test_dim1 = 10;
const unsigned int X_test_dim2 = 38;

const float X_test[10][38] = {
    -0.11024922321249885, -0.12470615670462065, -0.6867852074966913,
    0.7511112872365361, -0.007696997143860383, -0.004918644383724874,
    -0.08948642202040107, -0.09507567152556495, -0.8092618187059747,
    -0.011663642603760032, -0.036651869142258646, -0.024436507262009306,
    -0.01238515036740332, -0.02618002418454278, -0.018609896340735923,
    -0.04122119759327531, -0.0028174939213690777, -0.0975309439715147,
    -0.7170454919159579, -0.35434285002584825, -0.6372092679572258,
    -0.6319290328885425, -0.37436223991967527, -0.37443160310530493,
    0.7712831058493207, -0.349683030873482, -0.37455970440553465,

```

-1.402628854168409, -0.3852946548873903, -1.0051099793265696,
 0.03731130769783004, -0.25365662769412595, -0.02258683123335326,
 -0.6395319051152512, -0.6248707997445304, -0.32239374008094723,
 -0.3763870260680415, 0.21642633142264128, -0.11024922321249885,
 -2.468724163053645, 2.066113667991299, 0.7511112872365361,
 -0.0077310669071545745, -0.004918644383724874, -0.08948642202040107,
 -0.09507567152556495, -0.8092618187059747, -0.011663642603760032,
 -0.036651869142258646, -0.024436507262009306, -0.01238515036740332,
 -0.02618002418454278, -0.018609896340735923, -0.04122119759327531,
 -0.0028174939213690777, -0.0975309439715147, -0.7170454919159579,
 -0.3681102129968653, -0.6372092679572258, -0.6319290328885425,
 -0.37436223991967527, -0.37443160310530493, -0.36605989270614575,
 5.196207521885405, -0.37455970440553465, 0.7343425609306344,
 -0.9182555713235034, -1.0496586099715604, -0.3861460041139426,
 -0.28601951624008193, -0.28910340026287856, -0.6395319051152512,
 -0.6248707997445304, -0.38763462350750655, -0.3763870260680415,
 -0.21997021516886575, -0.11024922321249885, 2.219311849644404,
 -1.1761894520278895, 0.7511112872365361, -0.007754915741460509,
 -0.0048873108680200185, -0.08948642202040107, -0.09507567152556495,
 -0.8092618187059747, -0.011663642603760032, -0.036651869142258646,
 -0.024436507262009306, -0.01238515036740332, -0.02618002418454278,
 -0.018609896340735923, -0.04122119759327531, -0.0028174939213690777,
 -0.0975309439715147, 0.4793758683299075, 2.6194075517138344,
 -0.6372092679572258, -0.6319290328885425, -0.37436223991967527,
 -0.37443160310530493, 0.7712831058493207, -0.349683030873482,
 -0.33607292258588, 0.7343425609306344, 0.8341921878054109,
 0.6654636698605771, -0.3861460041139426, -0.4801968475158174,
 -0.28910340026287856, -0.6395319051152512, -0.6248707997445304,
 -0.38763462350750655, -0.3763870260680415, -0.6563667617603728,
 -0.11024922321249885, -0.12470615670462065, -0.44208308523109213,
 0.7511112872365361, -0.007722549466331027, -0.004642611031086857,
 -0.08948642202040107, -0.09507567152556495, 1.2356940323701657,
 -0.011663642603760032, -0.036651869142258646, -0.024436507262009306,
 -0.01238515036740332, -0.02618002418454278, -0.018609896340735923,
 -0.04122119759327531, -0.0028174939213690777, -0.0975309439715147,
 -0.7170454919159579, -0.35434285002584825, -0.6372092679572258,
 -0.6319290328885425, -0.37436223991967527, -0.37443160310530493,
 0.7712831058493207, -0.349683030873482, -0.37455970440553465,
 -0.6264647081182848, 1.2587542737799418, 1.0664013456654926,
 -0.43907816809041417, -0.4478339589698615, 0.155090881452997,
 -0.617048995298306, -0.6024325397096294, -0.38763462350750655,
 -0.3763870260680415, 0.6528228780141483, -0.11024922321249885,
 -0.12470615670462065, -0.44208308523109213, 0.7511112872365361,
 -0.007724593652128678, -0.003063998668432683, -0.08948642202040107,
 -0.09507567152556495, 1.2356940323701657, -0.011663642603760032,
 -0.036651869142258646, -0.024436507262009306, -0.01238515036740332,
 -0.02618002418454278, -0.018609896340735923, -0.04122119759327531,
 -0.0028174939213690777, -0.0975309439715147, -0.6471814708797029,
 -0.24420394625771188, -0.6372092679572258, -0.6319290328885425,
 -0.37436223991967527, -0.37443160310530493, 0.7712831058493207,
 -0.349683030873482, -0.37455970440553465, 0.7343425609306344,
 1.2587542737799418, 1.0664013456654926, -0.43907816809041417,
 -0.4801968475158174, -0.28910340026287856, -0.6395319051152512,
 -0.6248707997445304, -0.38763462350750655, -0.3763870260680415,

0.6528228780141483, -0.11024922321249885, -0.12470615670462065,
-0.6867852074966913, 0.7511112872365361, -0.007720505280533375,
-0.004918644383724874, -0.08948642202040107, -0.09507567152556495,
-0.8092618187059747, -0.011663642603760032, -0.036651869142258646,
-0.024436507262009306, -0.01238515036740332, -0.02618002418454278,
-0.018609896340735923, -0.04122119759327531, -0.0028174939213690777,
-0.0975309439715147, -0.7170454919159579, -0.35434285002584825,
-0.6372092679572258, -0.6319290328885425, -0.37436223991967527,
-0.37443160310530493, 0.7712831058493207, -0.349683030873482,
-0.37455970440553465, 0.7343425609306344, 0.021200959343543466,
-0.13641168174925328, -0.33321384013747096, 1.008496025598155,
-0.28910340026287856, -0.6395319051152512, -0.6248707997445304,
-0.38763462350750655, -0.3763870260680415, 0.21642633142264128,
-0.11024922321249885, -0.12470615670462065, -0.44208308523109213,
0.7511112872365361, -0.00771420237432395, -0.004851749814481967,
-0.08948642202040107, -0.09507567152556495, 1.2356940323701657,
-0.011663642603760032, -0.036651869142258646, -0.024436507262009306,
-0.01238515036740332, -0.02618002418454278, -0.018609896340735923,
-0.04122119759327531, -0.0028174939213690777, -0.0975309439715147,
-0.6733804787682985, -0.28550603517076306, -0.6372092679572258,
-0.6319290328885425, -0.37436223991967527, -0.37443160310530493,
0.7712831058493207, -0.349683030873482, -0.37455970440553465,
0.7343425609306344, 1.2587542737799418, 1.0664013456654926,
-0.43907816809041417, -0.4801968475158174, -0.28910340026287856,
-0.6395319051152512, -0.6248707997445304, -0.38763462350750655,
-0.3763870260680415, 0.6528228780141483, -0.11024922321249885,
2.219311849644404, -1.1761894520278895, 0.7511112872365361,
-0.007759344810688754, -0.004901236874999954, -0.08948642202040107,
-0.09507567152556495, -0.8092618187059747, -0.011663642603760032,
-0.036651869142258646, -0.024436507262009306, -0.01238515036740332,
-0.02618002418454278, -0.018609896340735923, -0.04122119759327531,
-0.0028174939213690777, -0.0975309439715147, -0.7257784945454897,
-0.35434285002584825, -0.6372092679572258, -0.6319290328885425,
-0.37436223991967527, -0.37443160310530493, 0.7712831058493207,
-0.349683030873482, 3.4741184775599305, -1.573989769530125,
1.0148569052413814, 1.0664013456654926, -0.43907816809041417,
2.756092007079775, -0.20026454391970344, -0.6395319051152512,
-0.6248707997445304, -0.38763462350750655, -0.3763870260680415,
0.6528228780141483, -0.11024922321249885, -0.12470615670462065,
2.127289198557699, -0.7362346401101137, -0.00776224074056876,
-0.004918644383724874, -0.08948642202040107, -0.09507567152556495,
-0.8092618187059747, -0.011663642603760032, -0.036651869142258646,
-0.024436507262009306, -0.01238515036740332, -0.02618002418454278,
-0.018609896340735923, -0.04122119759327531, -0.0028174939213690777,
-0.0975309439715147, 0.26105080259161084, -0.1615997684316096,
1.602663889932865, 1.6051037177847889, -0.37436223991967527,
-0.37443160310530493, -1.1849468516660815, 0.03852930781964016,
-0.37455970440553465, 0.7343425609306344, -0.9001890995799063,
-1.027384294649065, -0.06855302025511306, -0.4801968475158174,
-0.28910340026287856, 1.6087590765792643, 1.6189552037455606,
-0.38763462350750655, -0.3763870260680415, -0.6563667617603728,
-0.11024922321249885, -0.12470615670462065, 1.0873051789289023,
-0.7362346401101137, -0.00776224074056876, -0.004918644383724874,
-0.08948642202040107, -0.09507567152556495, -0.8092618187059747,

```

-0.011663642603760032, -0.036651869142258646, -0.024436507262009306,
-0.01238515036740332, -0.02618002418454278, -0.018609896340735923,
-0.04122119759327531, -0.0028174939213690777, -0.0975309439715147,
0.42697785255271625, -0.3681102129968653, 1.602663889932865,
1.6051037177847889, -0.37436223991967527, -0.37443160310530493,
-1.4806560312905028, -0.016929597707948767, -0.37455970440553465,
0.7343425609306344, -0.9814882224260931, -1.0942072406165508,
0.03731130769783004, -0.4801968475158174, -0.28910340026287856,
1.6087590765792643, 1.6189552037455606, -0.38763462350750655,
-0.3763870260680415, 0.6528228780141483
};

ytest10 = y_test[5:15].ravel()
print(c_writer.create_array(ytest10,"uint8_t","y_test"))

const unsigned int y_test_dim1 = 10;

const uint8_t y_test[10] = {
    1, 1, 1, 1, 1, 1, 1, 1, 0, 0
};

```